

Programming Assignment #1

Web Crawler

IAGO ZAGNOLI ALBERGARIA, Federal University of Minas Gerais, Brazil

Problem statement

The goal of this assignment is to implement a crawler capable of fetching a mid-sized corpus of web pages in a short time frame while respecting the politeness constraints defined by each crawled website. The crawler must initialize from a list of seed URLs and discover new links while adhering to politeness constraints defined by each target web page domain.

To fulfill the assignment requirements, the implementation must be written in Python 3.14 and, as the main target for this assignment, must crawl and store a total of 100,000 unique web pages. Also, the raw HTML content of the crawled pages must be packaged into compressed WARC files, with each file storing at most 1,000 web pages. Furthermore, the crawler must support a debugging mode that outputs a JSON-formatted record of each crawled page to standard output. Balancing parallelization with the necessity of avoiding server overloads (politeness) and memory optimization (revisitation) is the core technical challenge of this work.

Methods

To address the problem statement, the crawler was architected using a class-based approach, leveraging Python's `concurrent.futures` module for parallelization and several synchronized data structures to safely manage shared state across threads.

Data Structures

- **Frontier (Queue):** The URL frontier is implemented using a thread-safe queue. `Queue`, which operates as a First-In-First-Out (FIFO) buffer. Push and pop operations run in $O(1)$ time.
- **Revisitation State (Set):** A Python set, protected by a `threading.Lock`, tracks visited URLs. To minimize queue bloat, an early check is performed: discovered links are normalized (removing fragments and filtering parameters) and checked against the visited set *before* being added to the frontier.
- **Politeness Cache (Dictionary):** Two global dictionaries store the parsed robots.txt rules (using the `Protego` library) and the last access timestamp for each unique domain. This provides, on average, $O(1)$ lookups to quickly verify if a URL is allowed.

Crawling Policies

The crawler abides by the five core assignment policies:

- (1) **Selection Policy:** The crawler only follows discovered links with the MIME type `text/html`.
- (2) **Revisitation Policy:** Through the early check set mechanism, URLs are normalized and only added to queue if not visited yet.
- (3) **Parallelization Policy:** The process is parallelized across multiple threads using a `ThreadPoolExecutor`.
- (4) **Politeness Policy:** The crawler respects the robots exclusion protocol. In addition, threads calculate the necessary sleep time per domain and sleep *outside* the critical section to prevent bottlenecks. A default delay of 100ms is used, as asked in the PA document.

Code and data links:
https://github.com/iagozag/programmingSubjects/tree/master/information_retrieval/web_crawler
https://drive.google.com/drive/folders/1qQFE9d_9h3TA4wPwqWOK5AEt8GQhzU8?usp=sharing

Keywords: Web Crawling, Parallelization, Politeness, Information Retrieval

All design, implementation, debugging, and documentation were performed individually by the author for this assignment.

Author's Contact Information: Iago Zagnoli Albergaria, iagozag@ufmg.br, Federal University of Minas Gerais, Belo Horizonte, Minas Gerais, Brazil.

- (5) **Storage Policy:** The warcio library is used to package the raw HTML into gzip-compressed WARC files, rolling over every 1,000 pages.

Results

Empirical Efficiency

The efficiency of the parallelization strategy was evaluated by monitoring the download rate of 1000 pages in different thread configurations. With a single thread, the crawler is heavily limited by network latency and politeness delays, achieving an average download rate of **0.47** pages per second.

Upon increasing the pool to **40** threads, the download rate improved to **18.51** pages per second. However, the observed speedup is sub-linear. As the frontier becomes saturated with internal links from massive domains, threads frequently contend for the same domain locks and are forced to idle to respect the crawl delay, capping the maximum parallel throughput.

Corpus Characterization

An offline analysis script using warcio and BeautifulSoup was run against the generated corpus to extract the required statistics.

Domain Distribution: The crawler visited a total of **3683** unique domains. The distribution reflects the highly interlinked nature of the web, with some of the top domains being:

- portal.pucrs.br: **13414** pages (**13.41%**)
- www.gov.br: **12978** pages (**12.98%**)
- revistaseletronicas.pucrs.br: **10022** pages (**10.02%**)
- www.dell.com: **3147** pages (**3.14%**)
- www.lattes.cnpq.br: **1841** pages (**1.84%**)
- www.k-state.edu: **1183** pages (**1.18%**)
- www.youtube.com: **834** pages (**0.83%**)

Webpage Size Distribution: By stripping HTML tags and tokenizing the visible text by whitespace, the size distribution of the web pages is as follows:

- Maximum tokens on a page: **332898**
- Average (Mean) tokens per page: **2984.08**
- Median tokens per page: **1084.00**

Significance

This implementation highlights the trade-off in web crawling between pushing for more parallelism and still respecting politeness constraints. By organizing the code in a modular way and using an early check revisitation policy, the crawler was able to avoid the memory issues that often happen in simpler FIFO-based approaches. In the end, the generated WARC corpus and its analysis resulted in a clean and standardized dataset that can be used for downstream Information Retrieval tasks, while still meeting all the requirements of the assignment.

Use of AI Tools

AI-based tools were used only for debugging and general guidance. They helped mainly during the parallelization step, especially in deciding where to place locks to avoid race conditions, without noticeably hurting performance. All the code submitted is fully understood by the author.